

Comparaison entre le réseau de Pétri et l'exécution réelle

Dans cette partie, on compare le comportement du programme Scala/Akka avec le réseau de Pétri. L'objectif est de vérifier que le scénario observé dans le code correspond bien au chemin prévu par le modèle formel.

1. Idée générale

Le réseau de Pétri décrit les étapes principales d'un transfert réussi. Il montre comment une transaction passe du wallet à la mempool, puis au validator, ensuite à la base de données, et enfin au nettoyage de la mempool.

De son côté, le programme réel exécute exactement ces étapes sous forme de messages entre acteurs. Le log permet de voir chaque action dans l'ordre, ce qui rend la comparaison simple à faire.

2. Départ du scénario

Dans le scénario testé, Charlie crée une transaction vers Alice. On voit d'abord que le wallet de Charlie signe la transaction et l'envoie à la mempool. Cela correspond dans le réseau de Pétri aux premières étapes : recherche du wallet, création de la transaction, vérification du solde, puis signature et envoi.

Le log montre :

- la transaction est signée ;
- elle est envoyée à la mempool ;
- la signature est acceptée.

Dans le modèle, cela correspond aux transitions suivantes :

- `T_REGISTRY_GET_WALLET_FOUND;`
- `T_WALLET_CREATE_TX_START;`
- `T_WALLET_GET_BALANCE_REQUEST;`
- `T_DB_BALANCE_RESPONSE;`
- `T_WALLET_FUNDS_OK_BUILD_UNSIGNED;`
- `T_WALLET_SIGN_SEND_MEMPOOL;`
- `T_MEMPOOL_ADD_TX;`
- `T_MEMPOOL_VERIFY_OK_ADD_EMPTY.`

3. Passage dans la mempool

Une fois la transaction reçue, la mempool vérifie la signature. Comme elle est valide, elle ajoute la transaction dans sa file de priorité. Dans le log, on voit clairement que la transaction est ajoutée et que l'ordre est affiché selon les fees.

Dans le réseau de Pétri, cela correspond à la transition qui ajoute une transaction valide dans une mempool vide. Le modèle confirme donc que la transaction peut bien entrer dans la mempool à ce moment-là.

4. Minage et validation

Après quelques secondes, le validator interroge la mempool. Celle-ci envoie la transaction au validator, puis le minage commence. Le log montre ensuite que le bloc est miné avec succès, avec un nonce et un hash commençant par 0000, ce qui correspond à la condition de Proof of Work du projet, on peut évidemment la modifier, pour plus de difficulté.

Dans le réseau de Pétri, cette partie est représentée par :

- T_VALIDATOR_START_MINING;
- T_MEMPOOL_GET_TXS_NONEMPTY;
- T_DB_GET_LAST_BLOCK_REQUEST;
- T_DB_GET_LAST_BLOCK_RESPONSE;
- T_MINER_START;
- T_MINER_PROOF_OF_WORK.

Le code réel et le modèle formel suivent donc la même logique : récupération des transactions, récupération du dernier bloc, calcul du Proof of Work, puis envoi du bloc à la base de données.

5. Persistance et nettoyage

Quand le bloc est envoyé à la base de données, celle-ci confirme l'enregistrement. Le log montre que le bloc 0 est sauvegardé avec succès. Ensuite, le validator confirme le succès, prévient les wallets et demande le nettoyage de la mempool.

Dans le réseau de Pétri, cela correspond aux transitions :

- T_DB_SAVE_BLOCK_REQUEST;
- T_DB_SAVE_SUCCESS;
- T_VALIDATOR_CONFIRM_SAVED;
- T_MEMPOOL_REMOVE_CONFIRMED_ALL.

À la fin, la mempool est vide. Le log confirme cela aussi, car après le nettoyage, les cycles suivants montrent : "empty - nothing to send to Validator".

6. Correspondance globale

Ce test montre que le comportement réel suit bien le chemin prévu par le réseau de Pétri. Le modèle décrit donc correctement le scénario nominal d'un transfert réussi.

On peut résumer la correspondance ainsi :

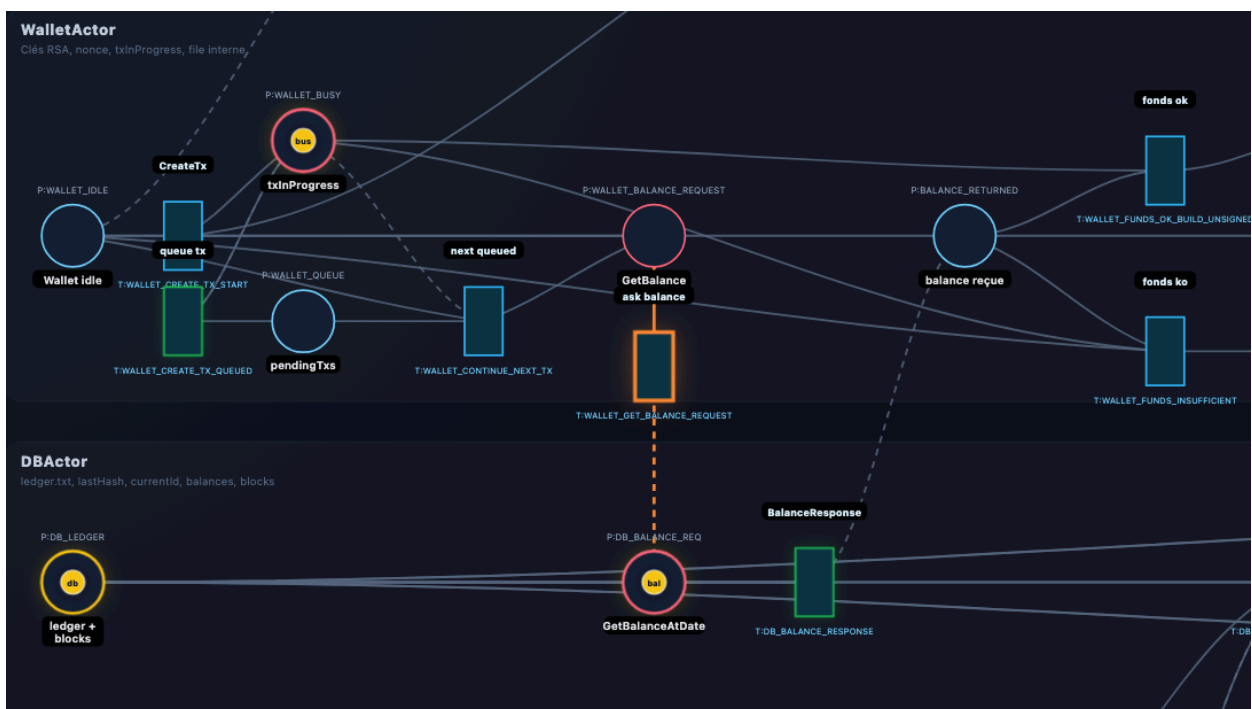
- le wallet prépare et signe la transaction ;
- la mempool vérifie et stocke la transaction ;
- le validator récupère la transaction ;
- le mineur calcule un bloc valide ;
- la base de données sauvegarde le bloc ;
- la mempool supprime les transactions confirmées.
-

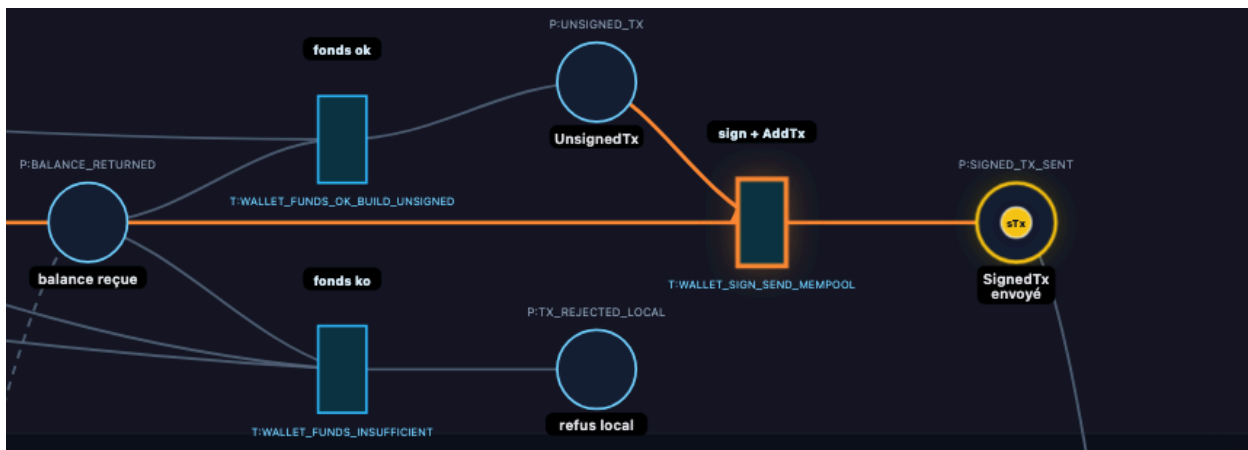
8. Limites

Cette comparaison reste une abstraction. Le réseau de Pétri ne calcule pas les vrais montants, les vrais hash, ni la cryptographie complète. Il sert surtout à représenter les étapes de contrôle et les états importants du système.

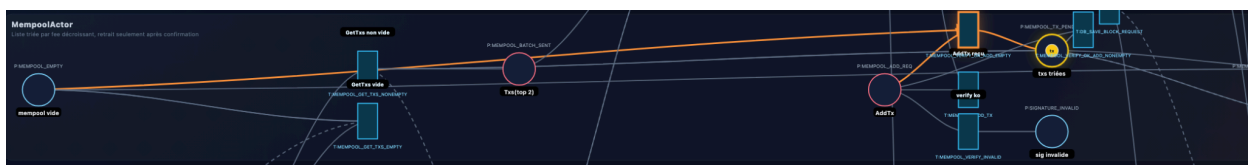
Cela reste suffisant pour analyser le comportement critique du projet et montrer que le code réel respecte bien la logique du modèle.

(Demande de la balance du compte envoyant, pour vérifier les fonds après)

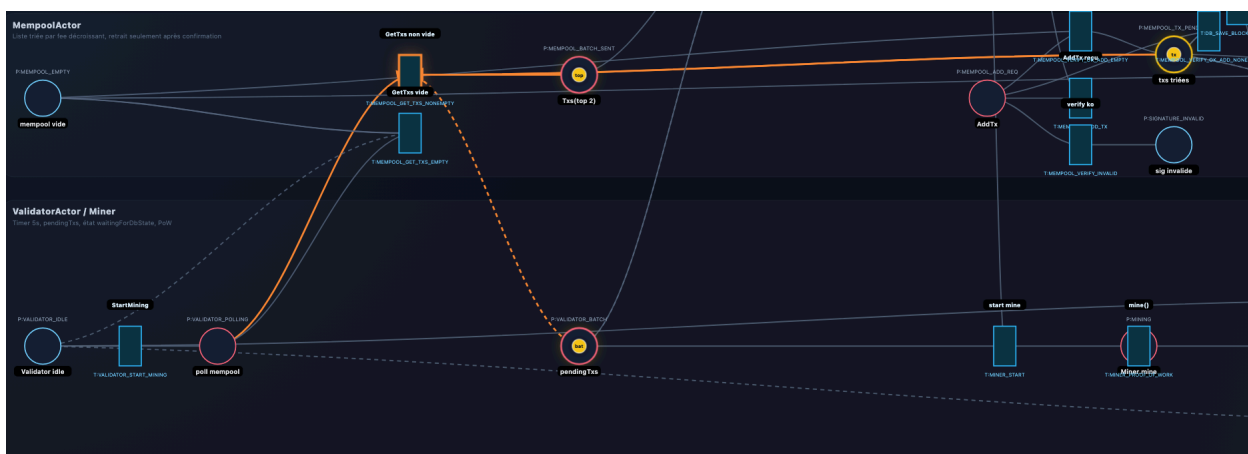




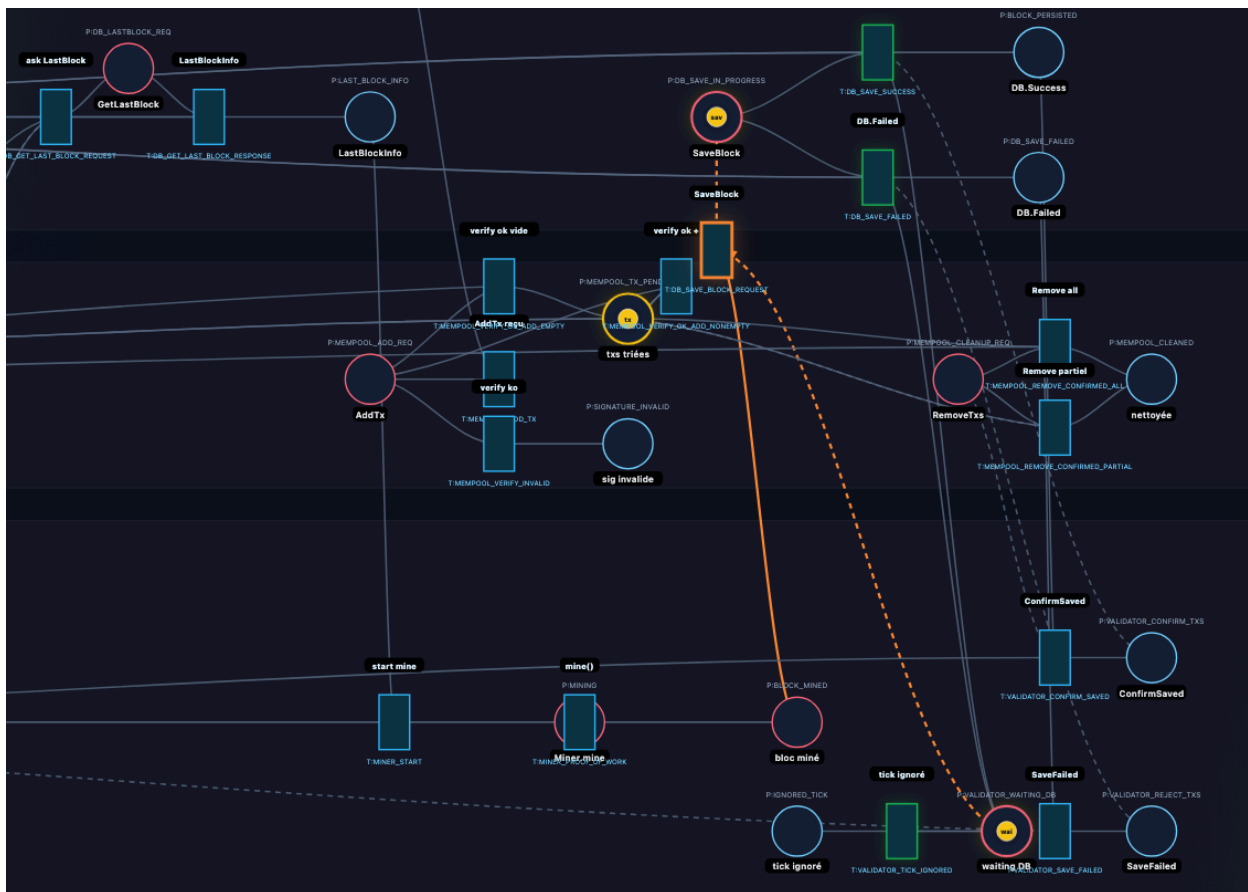
(les fonds sont ok, on est donc en train de signer, et envoyer la tx à la mempool)



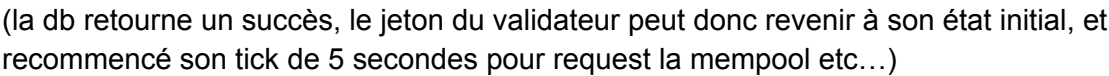
(la mempool a vérifié la signature, et la transaction est donc ajoutée à la priority queue)

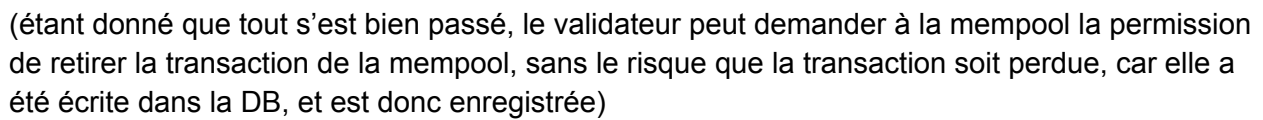


(en parallèle, le validator peut maintenant commencer à miner, car il a récupéré la transaction dans la mempool non vide)



(une fois le bloc miné, le validateur demande à la blockchain ou DB, la permission d'ajouter le nouveau block à la block chain => écrire la transaction)





(étant donné que tout s'est bien passé, le validateur peut demander à la mempool la permission de retirer la transaction de la mempool, sans le risque que la transaction soit perdue, car elle a été écrite dans la DB, et est donc enregistrée)